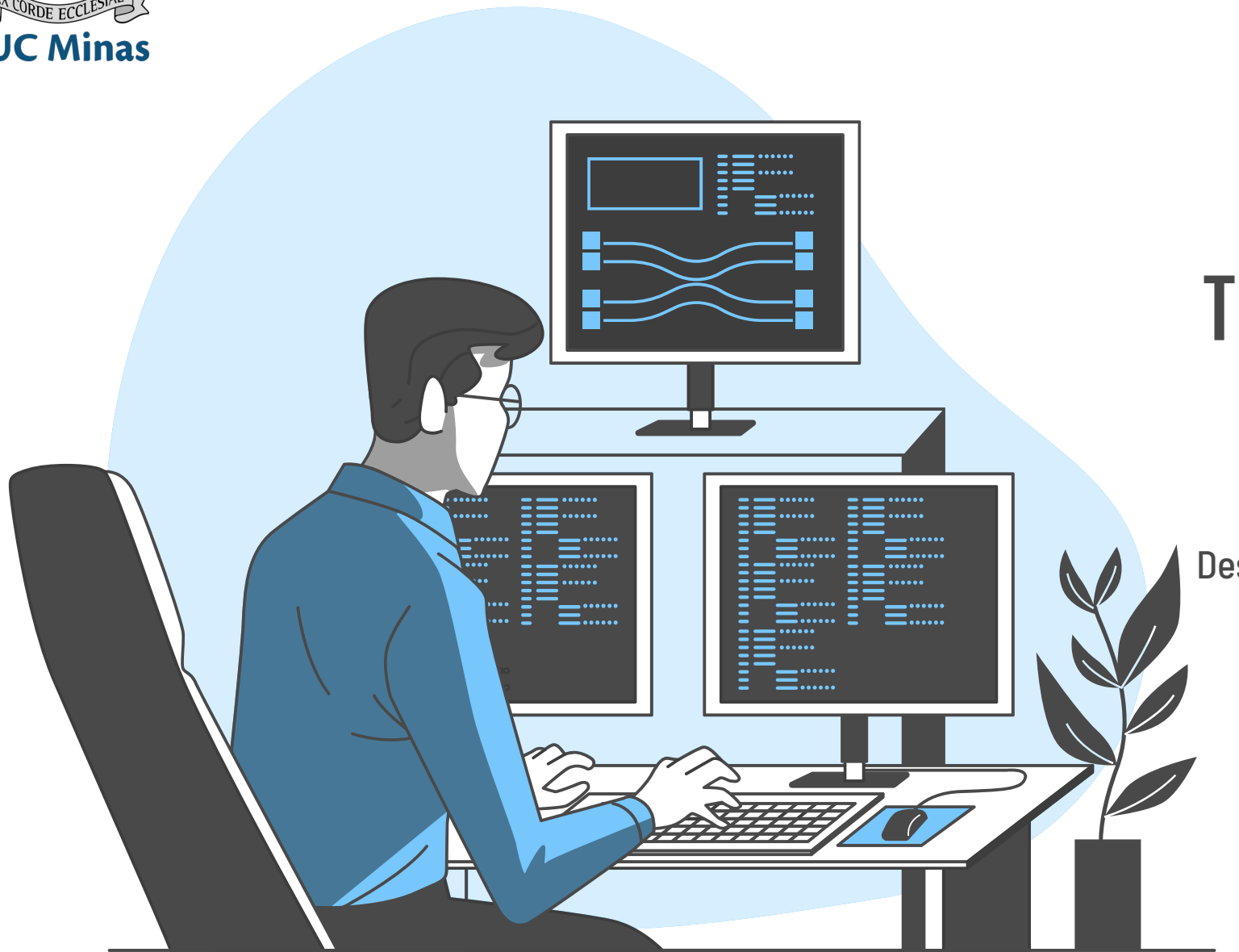
PUC Minas



Aula 08 · Back-end

Thymeleaf: páginas HTML geradas pelo Spring

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Entender a diferença entre **renderização no servidor** (Thymeleaf) e **SPA** (React)

2

Usar `@Controller` e `Model` para enviar dados à página

3

Escrever templates com `th:text`, `th:each`, `th:if`, `th:href` e `th:object`

4

Criar formulários que enviam dados para o Spring

5

Organizar `templates/` e `static/`

Antes de começar

- Aula 05: controllers

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Duas formas de fazer a tela

02 Onde ficam os arquivos

03 Expressões mais usadas

04 Formulários

05 Aprofundando: telas completas com Thymeleaf

01

Duas formas de fazer a tela

Duas formas de fazer a tela

	Thymeleaf (server-side)	React (SPA)
Quem monta o HTML	O Spring, a cada requisição	O navegador, com JavaScript
O controller devolve	Nome de um template	JSON
Anotação	`@Controller`	`@RestController`
Projetos	Um só projeto (`SecureLogin`, `SecureLoginPUC`)	Dois projetos: Back + Front
Deploy	Só o Render	Render + Vercel

Duas formas de fazer a tela

O **Thymeleaf** é um motor de templates: você escreve HTML normal com atributos especiais `th:*`, e o Spring substitui esses atributos pelos dados antes de mandar a página para o navegador. É ótimo para painéis administrativos, telas de login e projetos que não precisam de um front separado.

01

Duas formas de fazer a tela

Aula 08 · Back-end

Duas formas de fazer a tela

pom.xml

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

02

Onde ficam os arquivos

Onde ficam os arquivos

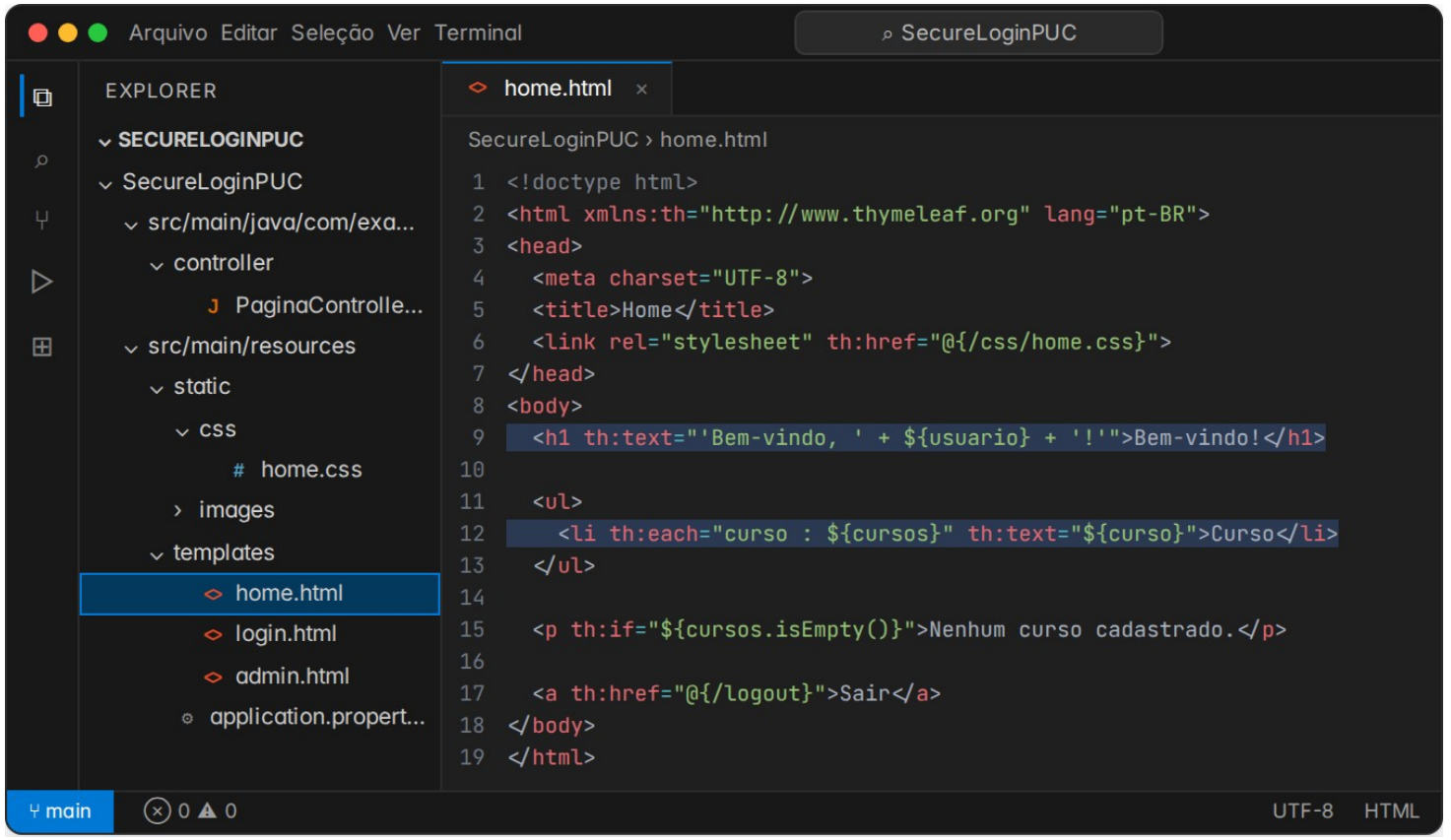
- `src/main/resources/templates/`: os templates `.html` processados pelo Thymeleaf.
- `src/main/resources/static/`: arquivos servidos como estão (`css/`, `js/`, `images/`). Um arquivo em `static/css/style.css` fica disponível em `http://localhost:8080/css/style.css`.

02

Onde ficam os arquivos

Aula 08 · Back-end

Onde ficam os arquivos



Template `home.html` com expressões do Thymeleaf. O texto entre as tags é só um exemplo, substituído pelo valor real.

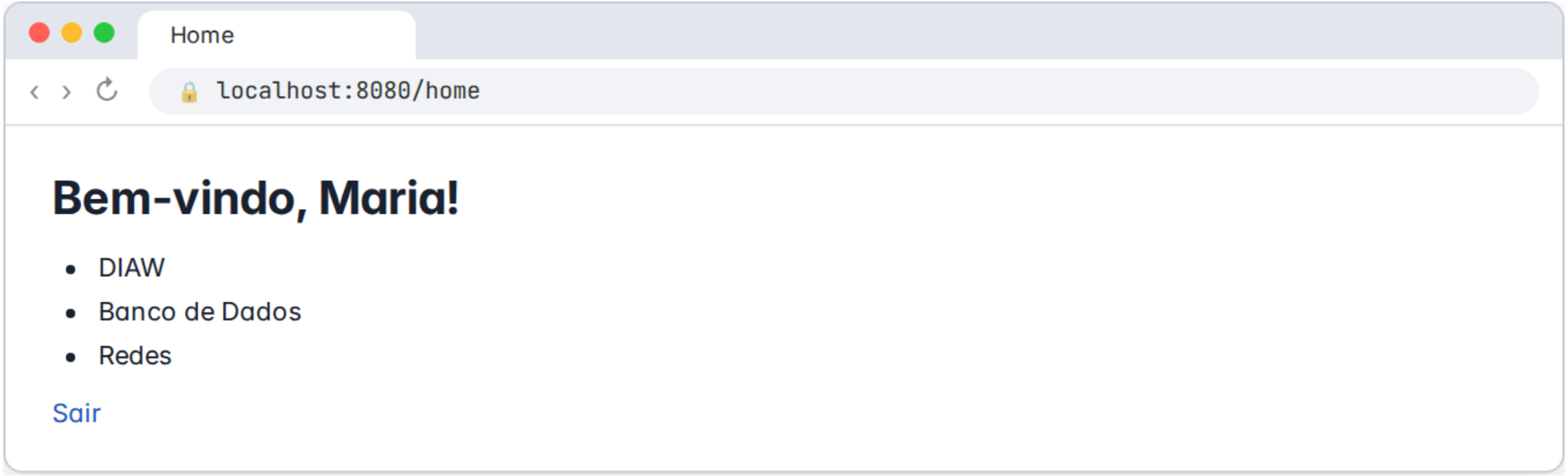
Onde ficam os arquivos

controller/PaginaController.java

```java
@Controller                                  // NÃO é @RestController
public class PaginaController {

    @GetMapping("/home")
    public String home(Model model) {
        model.addAttribute("usuario", "Maria");
        model.addAttribute("cursos", List.of("DIAW", "Banco de Dados", "Redes"));
        return "home";                       // → templates/home.html
    }
}
```

Onde ficam os arquivos



Página gerada pelo Spring: o navegador recebe HTML pronto, sem nenhuma expressão `th:`.

03

Expressões mais usadas

Expressões mais usadas

Atributo / sintaxe	Para que serve	Exemplo
`${...}`	Lê um atributo do `Model`	`${usuario}`
`@{...}`	Monta uma URL da aplicação	`@{/alunos/{id}(id=${a.id})}`
`*{...}`	Campo do objeto do formulário	`*{nome}`
`th:text`	Troca o texto da tag (com *escape* de HTML)	`th:text="${a.nome}"`
`th:each`	Repete a tag para cada item	`th:each="a : ${alunos}"`

Expressões mais usadas (cont.)

Atributo / sintaxe	Para que serve	Exemplo
`th:if` / `th:unless`	Mostra ou esconde	`th:if="${erro}"`
`th:href`, `th:src`	Links e imagens	`th:href="@{/css/app.css}"`
`th:object` + `th:field`	Liga um formulário a um objeto	ver abaixo
`th:replace` / `th:insert`	Reaproveita pedaços (*fragments*)	cabeçalho comum

04

Formulários

Formulários

O padrão é: um `GET` mostra o formulário vazio, um `POST` recebe os dados, salva e **redireciona** (padrão *Post/Redirect/Get*, que evita reenviar o formulário ao apertar F5).

Controller

```java
@GetMapping("/alunos/novo")
public String form(Model model) {
    model.addAttribute("aluno", new Aluno());
    return "aluno-form";
}

@PostMapping("/alunos")
public String salvar(@ModelAttribute Aluno aluno,
                     RedirectAttributes ra) {
    service.inserir(aluno);
    ra.addFlashAttribute("ok", "Aluno salvo!");
    return "redirect:/alunos";
}
```

templates/aluno-form.html

```html
<form th:action="@{/alunos}"
      th:object="${aluno}" method="post">
  <input th:field="*{nome}"
         placeholder="Nome">
  <input th:field="*{email}"
         type="email">
  <button type="submit">Salvar</button>
</form>

<p th:if="${ok}" th:text="${ok}"></p>
```

Formulários

Dica

Em desenvolvimento, coloque `spring.thymeleaf.cache=false` no `application.properties`. Assim, ao salvar o HTML, basta recarregar a página (com DevTools, nem precisa reiniciar).

Formulários

Observação

Com **Spring Security** no projeto, o Thymeleaf adiciona sozinho o campo oculto do **token CSRF** em todo `` `com method="post"` ``. Por isso, em projetos Thymeleaf, o CSRF fica **ligado**; em APIs REST com JWT ele é desligado (aula 09).

Formulários

Erro comum

`Error resolving template [home]`: o arquivo não está em `src/main/resources/templates/` ou o nome está diferente do `return`. **Página mostra o texto literal `home`**: você usou `@RestController` em vez de `@Controller`.

05

Aprofundando: telas completas com Thymeleaf

Fragmentos reutilizáveis

th:fragment e th:replace

```
<!-- templates/fragments/layout.html -->
<header th:fragment="cabecalho(titulo)">
  <h1 th:text="${titulo}">Título</h1>
  <nav>
    <a th:href="@{/home}">Início</a>
    <a th:href="@{/alunos}">Alunos</a>
  </nav>
</header>

<!-- em qualquer página -->
<div th:replace="~{fragments/layout :: cabecalho('Lista de alunos')}"></div>
```

Tabela com laço, estado e classes condicionais

templates/alunos.html

```
<table>
  <tr th:each="a, st : ${alunos}" th:classappend="${st.odd} ? 'zebra'">
    <td th:text="${st.count}">1</td>
    <td th:text="${a.nome}">Nome</td>
    <td th:text="${#temporals.format(a.criadoEm, 'dd/MM/yyyy')}">01/01/2026</td>
    <td th:switch="${a.status}">
      <span th:case="'ATIVO'" class="ok">Ativo</span>
      <span th:case="*" class="mut">Inativo</span>
    </td>
    <td><a th:href="@{/alunos/{id}/editar(id=${a.id})}">Editar</a></td>
  </tr>
</table>
```

Aprofundando: telas completas com Thymeleaf

Variável de estado (`st`)	Valor
`st.index`	Posição começando em 0
`st.count`	Posição começando em 1
`st.size`	Total de itens
`st.first` / `st.last`	Primeiro / último item
`st.odd` / `st.even`	Linha ímpar / par

Mostrando erros de validação

BindingResult + th:errors

```
@PostMapping("/alunos")
public String salvar(@Valid @ModelAttribute("aluno") AlunoForm form,
                     BindingResult erros) {
    if (erros.hasErrors()) return "aluno-form";   // volta com as mensagens
    service.inserir(form);
    return "redirect:/alunos";
}

<!-- no template -->
<input th:field="*{email}" th:classappend="${#fields.hasErrors('email')} ? 'invalido'">
<p th:if="${#fields.hasErrors('email')}" th:errors="*{email}">E-mail inválido</p>
```

Internacionalização

```
#{...} lê o texto do idioma do navegador
```

```
# src/main/resources/messages.properties
home.titulo=Bem-vindo, {0}!
# messages_en.properties
home.titulo=Welcome, {0}!

<!-- template -->
<h1 th:text="#{home.titulo(${usuario})}">Bem-vindo!</h1>
```

Thymeleaf + Spring Security

Formulário de login

`.formLogin(f -> f.loginPage("/login").permitAll())` usa o seu `login.html`.

Usuário na tela

Com `thymeleaf-extras-springsecurity6`: `sec:authentication="name"`.

Mostrar por papel

`sec:authorize="hasRole('ADMIN')"` esconde o link para quem não é admin.

CSRF automático

Todo `` já recebe o token oculto.

Aprofundando: telas completas com Thymeleaf

Observação

Os projetos `SecureLoginPUC`, `SecureLoginPUC_3` e `SecureLoginPUC_Matrix` são aplicações Thymeleaf completas: login, cadastro, recuperação de senha por e-mail, reCAPTCHA e área administrativa. Use-os como referência de organização de templates.

06

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Onde ficam os templates do Thymeleaf?

A src/main/resources/static

B src/main/resources/templates

C src/main/java

D public/

Resposta

Onde ficam os templates do Thymeleaf?

A src/main/resources/static

✓ src/main/resources/templates

C src/main/java

D public/

Por quê: `static/` guarda CSS, JS e imagens servidos como estão.

Pergunta

O que `th:each="a : ${alunos}"` faz?

A Esconde a tag

B Repete a tag para cada aluno

C Cria um link

D Valida o formulário

Quiz · questão 2 de 5

Resposta

O que `th:each="a : ${alunos}"` faz?

A Esconde a tag

✓ Repete a tag para cada aluno

C Cria um link

D Valida o formulário

Por quê: É o laço do Thymeleaf.

Quiz · questão 3 de 5

Pergunta

Para que serve `@{/css/home.css}`?

A Ler do Model

B Montar uma URL da aplicação

C Ler um campo de formulário

D Comentário

Resposta

Para que serve `@{/css/home.css}`?

A Ler do Model

✓ Montar uma URL da aplicação

C Ler um campo de formulário

D Comentário

Por quê: `${}` lê do Model; `*{}` lê campos do objeto do formulário.

Pergunta

O que é o padrão Post/Redirect/Get?

A Usar só GET

B Depois do POST, redirecionar para evitar reenvio no F5

C Enviar JSON

D Desligar o CSRF

Resposta

O que é o padrão Post/Redirect/Get?

A Usar só GET

✓ Depois do POST, redirecionar para evitar reenvio no F5

C Enviar JSON

D Desligar o CSRF

Por quê: `return "redirect:/alunos";`

Quiz · questão 5 de 5

Pergunta

A página mostra o texto literal `home` em vez do HTML. Causa provável:

A Falta o CSS

B Usou @RestController em vez de @Controller

C Porta errada

D Falta o banco

Resposta

A página mostra o texto literal `home` em vez do HTML. Causa provável:

A. Falta o CSS

✓ Usou @RestController em vez de @Controller

C. Porta errada

D. Falta o banco

Por quê: @RestController converte o retorno em resposta direta.

Fechamento

Resumo da aula

- ✓ Thymeleaf gera o HTML no servidor; o controller é `@Controller` e devolve o **nome do template**.
- ✓ Templates em `templates/`; CSS, JS e imagens em `static/`.
- ✓ `${}` lê do Model, `@{}` monta URLs, `*{}` lê campos do formulário.
- ✓ `th:text`, `th:each`, `th:if`, `th:href`, `th:object`/`th:field` resolvem a maior parte das telas.
- ✓ Use *Post/Redirect/Get* com `redirect:` e `RedirectAttributes`.

Exercícios

1. Crie a página `/alunos` que lista os alunos do banco em uma tabela HTML.

2. Crie o formulário de cadastro e redirecione para a lista com uma mensagem de sucesso.

3. Extraia o cabeçalho para `fragments/header.html` e use `th:replace` em todas as páginas.

4. Compare: o que mudaria para fazer a mesma tela em React consumindo `/api/alunos`?

Para ir além

- thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html
- Projetos `SecureLogin`, `SecureLoginPUC_3` e `HuggingfaceAPIWithUI`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 08?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web